

Retrieval Benchmark — Ablation Study

The Factory Floor — Industrial Maintenance Copilot · 25 August 2026

Why this study exists

The project already benchmarks *the system against a simpler alternative* — the agent versus a plain RAG baseline, in `notebooks/10_evaluation_baseline.ipynb`. That answers “*is my system better than the obvious?*”

This is the other benchmark: comparing **the project’s own design choices against each other**. `chunk_size=800`, `search_type="similarity"` and the reranker had all been in place since the beginning without any of them ever having been measured. An ablation answers “*why this configuration and not another?*”

Method

- **37 scenarios** from `eval_scenarios.csv`, the same set notebook 10 uses.
- **Zero LLM calls** (except the reranker arm). Every scenario carries an `expected_evidence_keywords` phrase individually verified to exist in the target chunk’s Cause/Remedy text *and* to be absent from the question itself, so a phrase match is a genuine chunk-level hit test — no generation, no judging, no cost, and no LLM noise masking the retrieval differences being measured.
- **One axis varied at a time**, not a full grid. A 3×3×2×3 grid would give 54 configurations, which is noise rather than information.

The two metrics

Metric	Question it answers	Definition
hit-rate@k	Did we find it at all?	Does <i>any</i> of the k retrieved chunks contain the expected phrase?
MRR	Did we find it at the top?	Mean of 1/position of the first correct chunk. 1st → 1.00, 2nd → 0.50, 5th → 0.20, absent → 0

Both are needed: two configurations can share a hit-rate while one leads with the answer and the other buries it fifth. The model weights what comes first, so only MRR sees that difference.

One deliberate exclusion. `code_aware` — the exact fault-code lookup — is **off throughout**. With it on, the 17 `vfd_fault_code` scenarios out of 37 would score perfectly under every configuration and flatten the differences this study exists to measure. Its own gain is quantified separately: definition page 18% → 100%.

Axis A — chunk size

Configuration	Chunks	hit-rate@5	MRR
chunk_size = 400	23,336	64.9%	0.433
chunk_size = 800 (<i>current</i>)	12,301	78.4%	0.596
chunk_size = 1600	6,754	81.1%	0.610

Per category — the finding is mechanical, not mysterious

Category	n	400	800	1600
general	3	67% / 0.19	67% / 0.40	100% / 0.57
motor_manual	4	75% / 0.55	75% / 0.62	75% / 0.62
motor_scenario	10	80% / 0.50	80% / 0.62	80% / 0.68
vfd_fault_code	17	59% / 0.43	82% / 0.63	82% / 0.62
vfd_general	3	33% / 0.33	67% / 0.50	67% / 0.33

Small chunks hurt fault codes most — 82% down to 59% at 400 characters. The reason is concrete: a fault entry's *Cause* and *Remedy* end up split into different chunks, so retrieving one loses the other.

Axis B — search strategy

Configuration	hit-rate@5	MRR	Chunks returned
similarity	81.1%	0.610	5.0
mmr (lambda_mult = 0.5)	73.0%	0.591	5.0
similarity_score_threshold (0.2)	62.2%	0.452	3.8

The threshold row is invalid, not merely worse. The collection is built with `hnsf.space='l2'`, so LangChain resolves relevance through its Euclidean function, which here returned scores **outside 0–1**, **several of them negative** (literal warning: *“Relevance scores must be between 0 and 1”*). The threshold therefore discarded chunks irrespective of relevance. That row measures a broken metric, not a worse strategy. Testing thresholding properly would require rebuilding the collection in cosine space — which is also the route to fixing difficulty #2 (no relevance floor).

Axis C — the reranker

Configuration	hit-rate@5	MRR	Time
reranker OFF	81.1%	0.610	8 s
reranker ON	86.5%	0.685	45 s

Is the reranker just “fetch more”?

The reranker pulls 15 candidates and returns the best 5. Plain `k=10` also pulls more, so a control is needed to tell the two apart:

Configuration	hit-rate	MRR	Chunks the model reads
reranker ON (k = 5, fetches 15)	86.5%	0.685	5
plain k = 10, no reranker	86.5%	0.619	10

Identical coverage, +0.066 MRR, half the context. The LLM call is buying *ordering*, not *coverage* — which is the justification the reranker never had before.

Axis D — k

Configuration	hit-rate	MRR
k = 3	75.7%	0.599
k = 5 (current)	81.1%	0.610
k = 10	86.5%	0.619

Hit-rate rises almost mechanically with k — more chunks, more chances to contain the phrase — while MRR barely moves. Raising k alone buys coverage at the cost of diluting the context the model reads.

Combined vs. current

An ablation measures each axis in isolation; stacking the winners assumes the effects add up, which is not guaranteed. So the combination was measured directly rather than inferred:

Configuration	hit-rate@5	MRR
CURRENT production — 800 / similarity / k=5	78.4%	0.596
COMBINED — 1600 / similarity / k=5 / reranker	86.5%	0.685

+8.1 percentage points hit-rate, +0.088 MRR.

What was *not* changed, and why

Production still runs `chunk_size=800`. Adopting 1600 means rebuilding the main vector store and re-running notebooks 02–10, which is a separate decision with its own cost — not something to fold silently into a benchmark.

Limitations — read these before quoting any number above

- **A phrase match is a proxy for relevance, not relevance itself.** A chunk can contain the expected phrase and still be the wrong page; a genuinely useful chunk phrased differently counts as a miss. Same limitation already disclosed for notebook 10's keyword scoring, applied one layer earlier in the pipeline.
- **37 scenarios is a small sample.** Differences of a few percentage points are within noise; only the large gaps should drive a decision.
- **The per-category breakdown is smaller still** — `general` and `vfd_general` have 3 scenarios each, so those figures move in 33% steps and must not be read as trends.
- **This measures retrieval, not answers.** Better retrieval should mean better answers, but that link is assumed here, not demonstrated.
- **`code_aware` is off throughout**, so the fault-code figures describe the semantic path only. The shipped app resolves those codes by exact lookup instead.

Generated from `notebooks/11_retrieval_benchmark.ipynb`, executed end to end on 25 August 2026. Variant stores (`data/vectorstore_cs400`, `data/vectorstore_cs1600`) are gitignored and rebuildable in roughly 3 minutes each.